

# **23CS004 – PROGRAMMING WITH DATA STRUCTURES**

## **QUESTION BANK**

### **MODULE I – INTRODUCTION TO LINEAR DATA STRUCTURES**

#### **PART A – 2 MARK QUESTIONS**

- Define data structure.
- List the types of data structures.
- Differentiate between static and dynamic data structures.
- Compare arrays and linked lists.
- What is a singly linked list?
- Write any two advantages of linked lists over arrays.
- Define a node in a linked list.
- What is the difference between singly and doubly linked lists?
- Define circular linked list.
- What are the applications of circular linked lists?
- Define stack.
- What are the basic operations of a stack?
- Give any two real-life applications of stacks.
- Define queue and its operations.
- What is the difference between stack and queue?
- Define infix, prefix, and postfix expressions.
- What is meant by stack overflow and underflow?
- Define priority queue.
- Write any two applications of queue.
- What is the use of linked list implementation in stack or queue.

#### **PART B – 16 MARK QUESTIONS**

##### **1. THEORY QUESTIONS:**

- Explain in detail the different types of data structures with examples.
- Compare arrays and linked lists. What are the advantages and disadvantages of each?
- Explain the structure, operations, and applications of singly linked lists with diagrams.
- Discuss in detail the operations on doubly linked list with algorithms.
- Explain the concept and advantages of circular linked list with applications.
- Discuss stack ADT (Abstract Data Type) and its operations using both array and linked list implementations.
- Explain various applications of stack in expression conversion and evaluation.
- Explain queue and its types. Discuss array and linked list implementations of queues.
- Write short notes on: Priority Queue and Applications of Stack and Queue.

## **2. PROBLEM ORIENTED QUESTIONS:**

- Write a program to create a singly linked list and perform insertion and deletion operations.
- Implement a doubly linked list with insertion at the beginning, end, and specific position.
- Write a program to convert an infix expression to postfix expression using stack.
- Write a program to evaluate a postfix expression using stack.
- Implement a queue using an array and perform enqueue and dequeue operations.
- Implement a circular queue using array or linked list.
- Write a program to check whether parentheses are balanced in an expression using stack.

## **MODULE II – NONLINEAR DATA STRUCTURES**

### **PART A – 2 MARK QUESTIONS**

- Define a tree.
- Define a binary tree.
- What is a binary search tree (BST)?
- What are the applications of trees?
- Define preorder, inorder, and postorder traversals.
- Differentiate between complete and full binary trees.
- What is a leaf node?
- Define a graph.
- Differentiate between directed and undirected graphs.
- What is an adjacency matrix?
- What is an adjacency list?
- What is a weighted graph?
- Define BFS and DFS.
- What are the applications of graphs?
- Write the time complexity of BFS and DFS algorithms.
- Define level order traversal.
- Mention any two real-life applications of graphs.
- Define edge, vertex, and degree in graphs.
- What is meant by path and cycle in a graph?
- What is the difference between tree and graph?

### **PART B – 16 MARK QUESTIONS**

#### **3. THEORY QUESTIONS:**

- Explain the different types of trees with examples.
- Explain the structure and properties of binary trees.
- Describe binary tree traversal techniques with algorithms and examples.
- Explain in detail the creation, insertion, and deletion operations in a binary search tree.
- Differentiate between binary tree and binary search tree with examples.
- Explain adjacency matrix and adjacency list representations of graphs.
- Explain breadth-first search (BFS) and depth-first search (DFS) traversal algorithms with examples.
- Discuss various types of graphs (directed, weighted, cyclic, acyclic) with real-world examples.

#### **PROBLEM ORIENTED QUESTIONS:**

- Construct a binary tree from given inorder and preorder sequences and show all traversals.
- Write a program to insert and delete nodes in a binary search tree.
- Implement inorder, preorder, and postorder traversal of a binary tree using recursion.

- Implement BFS and DFS for a given graph using adjacency list.
- Write a program to represent a graph using adjacency matrix and perform traversal.
- Given a set of nodes, create a binary search tree and display the tree traversals.
- Design a C++ program to perform level order traversal in a binary tree.

## **MODULE III – HASH FUNCTIONS, SORTING AND SEARCHING TECHNIQUES**

### **PART A – 2 MARK QUESTIONS**

- Define hashing.
- What is a hash function?
- Define collision.
- What is linear probing?
- Define quadratic probing.
- What is double hashing?
- Differentiate between open addressing and chaining.
- What is a direct address table?
- Define sorting.
- Define searching.
- What is the time complexity of bubble sort?
- What is the difference between selection sort and insertion sort?
- Define merge sort.
- Define quick sort.
- What is the pivot element in quick sort?
- What is binary search?
- Define linear search.
- Mention the best and worst case of binary search.
- Write any two applications of hashing.
- Mention one advantage of merge sort over quick sort.

### **PART B – 16 MARK QUESTIONS**

#### **4. THEORY QUESTIONS:**

- Explain the concept of hashing and various hash functions with examples.
- Discuss collision resolution techniques — linear probing, quadratic probing, and double hashing.
- Explain in detail different sorting techniques and their time complexity analysis.
- Compare bubble sort, selection sort, and insertion sort.
- Explain merge sort algorithm with suitable example and complexity analysis.
- Explain quick sort algorithm step by step with an example.
- Compare merge sort and quick sort in terms of performance and memory usage.
- Explain searching techniques — linear search and binary search with examples.

#### **5. PROBLEM ORIENTED QUESTIONS:**

- Implement hashing using linear probing for a given set of keys.
- Write a program for quadratic probing and double hashing methods.
- Implement bubble sort, selection sort, and insertion sort in C++ and show outputs.
- Implement merge sort and show step-by-step merging process.
- Implement quick sort and trace the algorithm for a given array.

- Write a program for binary search on a sorted array.
- Write a program for linear search and calculate number of comparisons.
- Given an unsorted array, demonstrate sorting using selection sort and show intermediate steps.